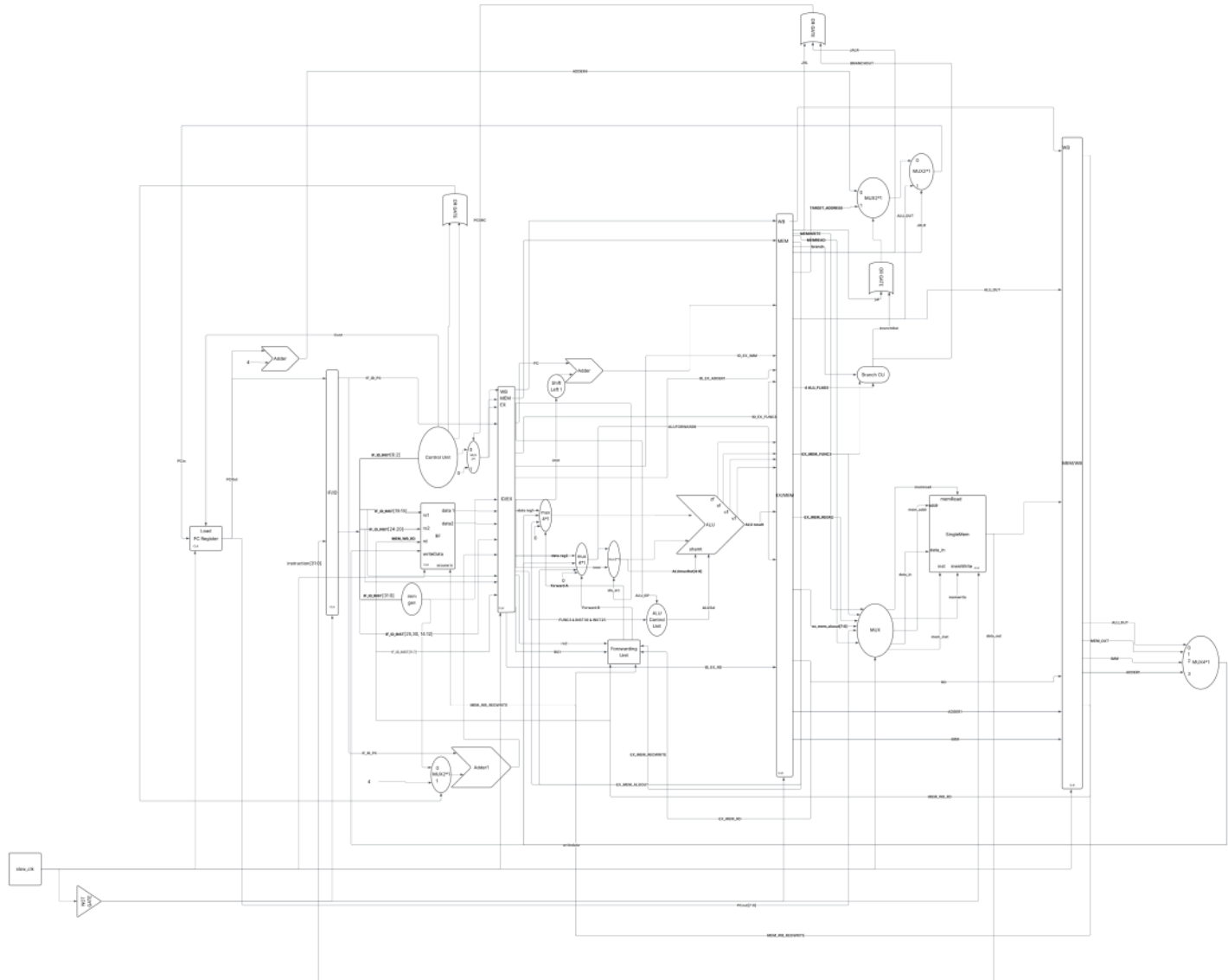


Report

Monday, November 24, 2025 8:26 PM

The manual schematic design

The link for a better resolution: https://lucid.app/lucidchart/3f6f110c-cd35-4564-a0a3-783926afffa4/edit?viewport_loc=1089%2C494%2C3230%2C1221%2C0_0&invitationId=inv_23ee3e54-9c40-419e-8256-65c489ea2050

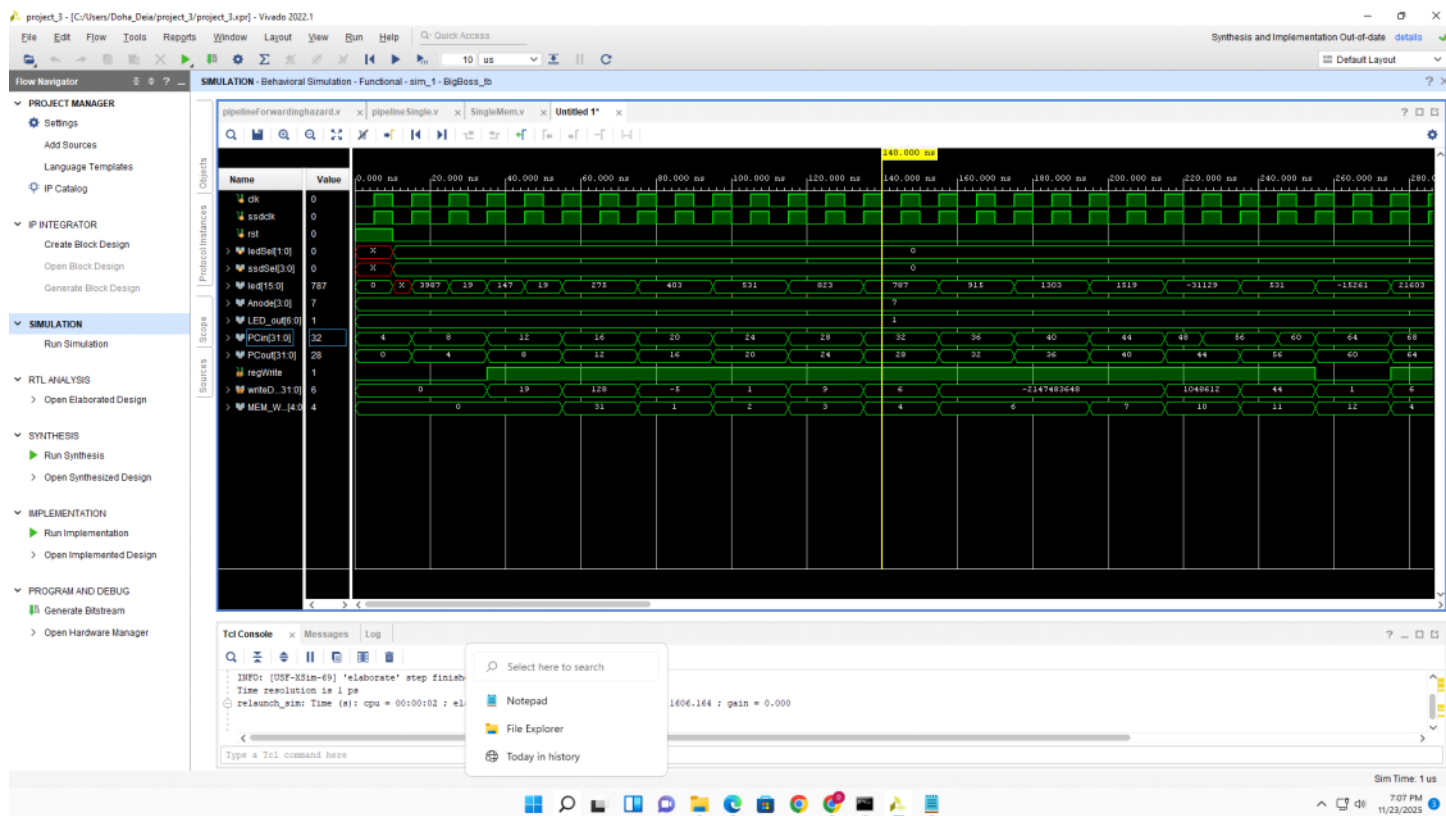
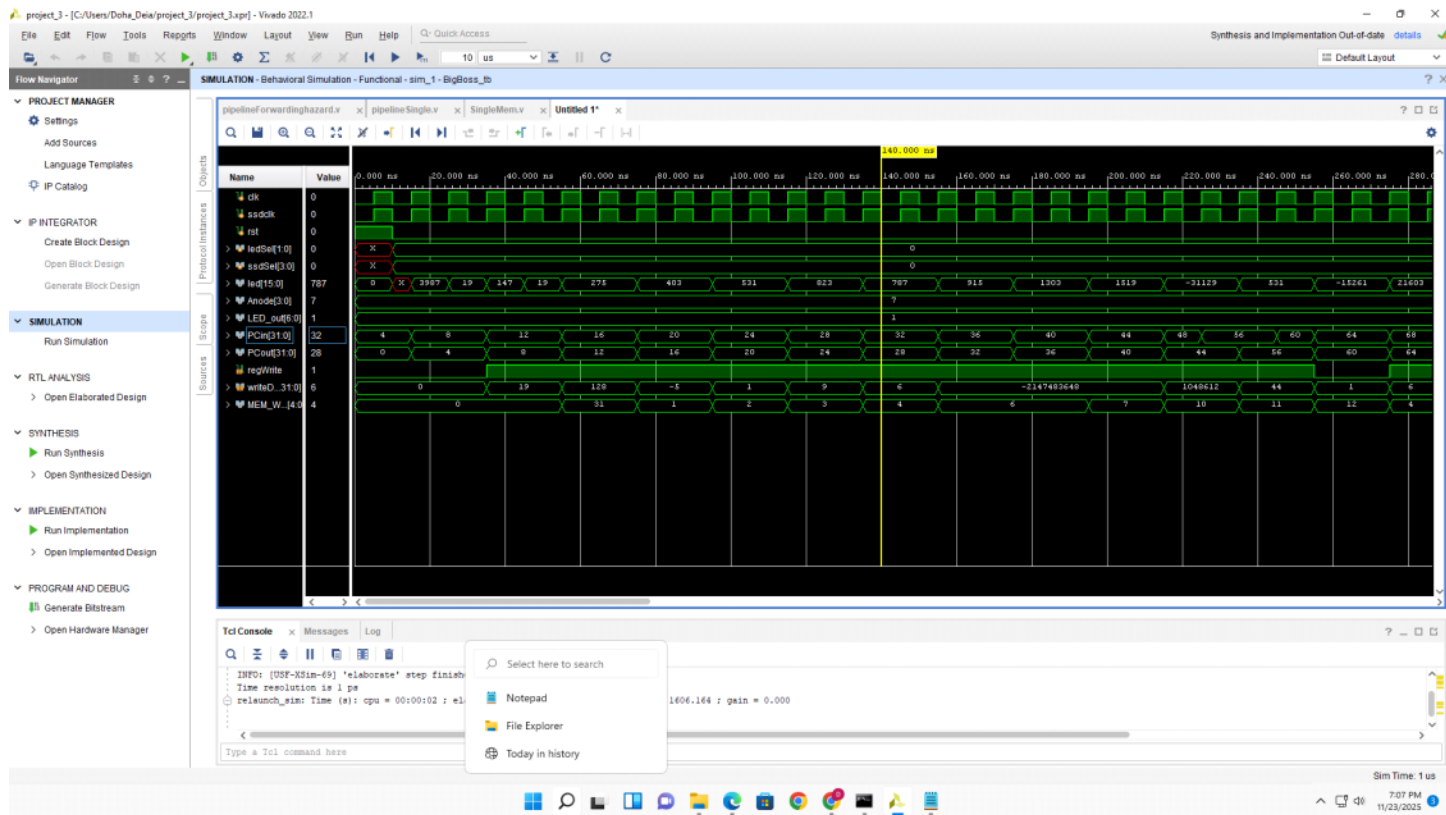


Some screenshots of the tests

The following test result is from the screenshot "test1.1" in the Tests Screenshot folder of the test_1 file in the Testcases folder

The results show the writeback data handled correctly for some instructions (i.e. ADDI, LUI, AUIPC, JAL, branch instructions)





The following test result is from the screenshot "test1.2" in the Tests Screenshot folder of the test_2 file in the Testcases folder

The results show the writeback data handled correctly for the load/store instructions

- Three pipeline modules were developed: basic pipeline, pipeline with forwarding, and pipeline with forwarding+flush
- The final implemented version uses the pipeline with forwarding and flush mechanisms with no need for the hazard detection in this implementation project
- Single memory implementation integrated into the pipelined design

Single Memory

- Unified memory module for both instructions and data
- Implements byte-addressable memory with proper load/store handling
- Resolve structural hazards through clock division and memory arbitration (using slow clk and normal clk)

Issues encountered & solutions

Issue	Result	Reason	Solution
Flush 3 consecutive instructions	Many instructions are not written	Fetching is done every 2 cycles not 1	Flush only one instruction
Using Hazard detection unit	Some instructions are not written	Fetching is done every 2 cycles not 1	Not using hazard detection in the main program
Wrong results in simulation after pipelining	The outputs are wrong and the PC counter is wrong	- Missing some connections in the path - Wrong numbering for in/out the registers while pipelining	- Recalculating the numbers of in/out wires - Ensuring the correct path

Bonus Features

1. Support for all integer multiplication and division instructions

Supported in: ALU0Flag and ALU control unit

- **ALU0Flag** module modified to support M-extension instructions (multiplication and division)
- **ALU** control unit enhanced to handle all func3 and func7 combinations for RISC-V instruction types
- **Multi** and **DivRem** modules implemented and integrated to support M-extension bonus feature

2. A Test program generator

- Generates 0-31 instructions randomly based on the number of instructions the user wants
- Creates organized test folders containing assembly, binary, hex, and instruction detail files for processor verification and simulation.
- Implements little-endian byte ordering, memory bounds checking, and produces Verilog-compatible memory initialization files.
- Ensures realistic instruction sequences with register initialization and preserves x31 as memory base address throughout execution.
- Here is a sample screenshot of the result in the terminal

```

EXPLORER
  BONUS TEST GENERATOR
    .vscode
    Boss01_test
    Boss02_test
    Boss03_test
    Boss04_test
    Boss05_test
    Boss06_test
    Boss07_test
    testmain.cpp
    testmain.exe
    To run the program.txt

testmain.cpp
To run the program.txt

To run the program.txt
1 To run the program:
2 1. Open the terminal command prompt
3 2. type: g++ -o testmain.exe testmain.cpp
4 3. Enter
5 4. Run: testmain.exe
6 5. Enter
7 6. have a nice testing generator process 😊
8

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
D:\Courses AUC\Fall 25\Comp. Architecture\Project 1\submission milestone3\Bonus Test Generator>testmain.exe
=====
RISC-V TEST PROGRAM GENERATOR
=====
Features:
- Fixed initialization: ADDI x0,x0,0 and ADDI x31,x0,128
- x31 preserved at 128 for memory operations
- All instruction types: R, I, S, B, U, J formats
- Instruction limit: 0-31
  
```

